Strathmore University

School of Computing and Engineering Sciences

Lab Report — IoT Environmental Monitoring with TTGO, DHT22, MQTT & SQLite

Project Title: IoT Environment Monitoring System

BICS: Embedded Systems and IoT

Lecturer: Mr. Solomon Itotia

Date: 11/06/2026

Repository: https://github.com/quantanmreaper/MicroPython-Lab-ICS-4D-Group-1.git

Group 1

167077 - Mepani Bhavin Ramesh

161088 - Muhia Robby Mwangi

169649 - Kimathi Austin Muthomi

167999 - Samuel Mwesigwa

166529 - Hope Wanjohi

166991 - Omwanda Philip Maxwell

1. Abstract

This project implements a complete Internet of Things (IoT) pipeline which measures temperature and humidity using a DHT22 sensor connected to a TTGO LoRa32 board. The board runs MicroPython, connects to WiFi, and publishes the readings as JSON messages over the MQTT protocol to a public broker (broker.hivemq.com). A Python subscriber running on our PC receives these messages and stores each reading, with a timestamp, into an SQLite database. The system was first prototyped in the Wokwi browser simulator and then deployed to physical TTGO hardware.

2. Objectives

Our system was required to:

a) Use **MicroPython** as the firmware language.

b) Use a **TTGO** board as the target hardware.

c) Read **temperature** and **humidity** from a **DHT22** sensor.

d) Connect to a network over **WiFi**.

e) **Publish** the data through **MQTT**.

f) Send the data as **JSON** payloads.

g) **Receive** the messages on a **PC subscriber**.

h) **Store** readings into an **SQLite** database.

i) Produce **screenshots** and **documentation**.

j) Publish the **source code to GitHub**

3. Hardware Note

Our lecturer, Mr. Itotia instructed us to use TTGO, instead of the specified generic ESP32. It still uses the same approach, but we had to make sure to check the datasheet before flying them and use WiFi for data transmission. We thus used the the **TTGO LoRa32** as the target board. The main difference between the two was the **pin map:** on the TTGO, we used GPIO23 directly addressing the lecturer's note.

4. System Architecture

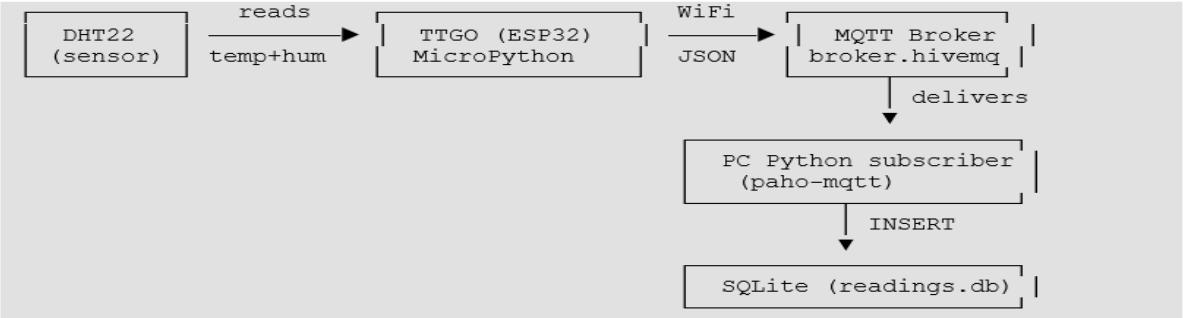


Figure 1 System Architecture

Data flow:

1. The DHT22 measures temperature and humidity.
2. The TTGO (MicroPython reads both values from the sensor's DATA pin.
3. The values are packed into a JSON string, e.g. {"temperature": 24.5, "humidity": 60.0}.
4. The board connects to WiFi and publishes the JSON to the topic *university/group1/dht22* via the MQTT broker
5. The broker forwards the message to all subscribers of that topic.
6. The PC subscriber receives the JSON, parses it, and inserts a timestamped row into the SQLite database.

5. Components & Tools

Table 5.1 Components and Tools

Component	**Role**	**Notes**
TTGO LoRa32 (ESP32)	Microcontroller / "brain"	WiFi used; LoRa unused
DHT22	Temperature + humidity sensor	3 connections only
MicroPython	Firmware language on the board	umqtt.simple, dht, network
WiFi	Data transport	ESP32 is 2.4 GHz only
MQTT	Messaging protocol	publish/subscribe
HiveMQ broker	Message router	broker.hivemq.com:1883 (public)
JSON	Payload format	json.dumps / json.loads
paho-mqtt	PC MQTT client library	pip install paho-mqtt==1.6.1
SQLite	Database	single file readings.db

Component	Role	Notes
Wokwi	ESP32 simulator	used for prototyping
PyCharm + mpremote	Editor + tool to upload/run on the board	python -m mpremote over USB

6. Methodology

Wiring: The DHT22 sensor was connected to the TTGO ESP32 using 3.3 V (VCC), GND, and GPIO23 (DATA). During simulation, GPIO15 was used in Wokwi.

Firmware: A MicroPython program connected the board to WiFi and an MQTT broker, read temperature and humidity from the DHT22 sensor every 5 seconds, formatted the readings as JSON, and published them to an MQTT topic.

Data Storage: A Python MQTT subscriber running on the laptop received the sensor readings and stored them in an SQLite database together with timestamps for later retrieval and analysis.

Simulation and Prototype: We first simulated the solution on wokwi and then went on ahead to build the actual physical hardware and IoT solution.

Prototype: We soldiered the TTGO board ourselves in order to for the pins to be used for connection

7. Results

The system was able to successfully transmit environmental data from the DHT22 sensor to the SQLite database through MQTT. The Wokwi simulation we had done first verified correct WiFi connectivity, MQTT communication, and JSON message publishing before deployment to physical hardware. On the TTGO board, live temperature and humidity readings were received and stored successfully. Changes in environmental conditions, such as breathing near the sensor, produced immediate increases in humidity values, confirming end-to-end functionality.

	id	temperature	humidity	timestamp
1	1	24	40	2026-06-10 11:49:15
2	2	24	40	2026-06-10 11:49:20
3	3	24	40	2026-06-10 11:49:26
4	4	24	40	2026-06-10 11:49:31

Figure 2 A portion of data received

8. Testing and Debugging

Some of the problems we encountered are explained clearly in the table below

Table 8.1 Testing and Debugging

Problems	Cause	Solution
Sensor read error timeout (Err no 116)	The sensor we were working with was faulty	We replaced the DHT22 sensor with a different sensor which resolved the error
Sensor read/publish error (Err no 116)	Incorrect wiring / Pin configuration	After changing pins from GPIO4 to GPIO 23, this error was resolved.

9. Conclusion

The project successfully implemented the IoT system using the TTGO board, DHT22 sensor, MQTT communication, and SQLite storage. The sensor readings were collected as needed, transmitted wirelessly, and stored for analysis, showing the practical application of IoT technologies. This was a good project for our group as it has helped us as well in understanding the process of communication in IoT. It will also help us significantly in better developing our semester project.

10. Appendix

Below, Figure 3, is an image our group while working on the groupwork.

Figure 3 Group 1

Below ,is an error we got when publishing the data , as the process was timing out but we fixed the code to handle out the error and managed to fix it.

```
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:4200
load:0x40078000,len:15236
load:0x40080400,len:3332
entry 0x400805a8
BOOT TEST: running latest TTGO code
Configured WiFi SSID: ESP32TEST
Configured DHT pin: 4
Connecting to WiFi: ESP32TEST
WiFi status: 1001
WiFi status: 1001
WiFi status: 1001
WiFi status: 1001
WiFi status: 202
WiFi status: 202
WiFi connected.
Network config: ('10.81.156.86', '255.255.255.0', '10.81.156.134', '10.
Connected to MQTT broker: broker.hivemq.com
Starting publish loop. Topic: b'university/group1/dht22'
Error during read/publish: [Errno 116] ETIMEDOUT
Connected to MQTT broker: broker.hivemq.com
Error during read/publish: [Errno 116] ETIMEDOUT
```

Figure 4 Publish and Read Error

Below image shows the error we were getting when the dht22 was not reading/sensing the data and sending/publishing it to the database. So we changed the faulty dht22 sensor as per the lecturers advice and also change the data pin which was previously connected to pin 4 on TTGO board to pin 23.

Figure 5 Sensor Read Error

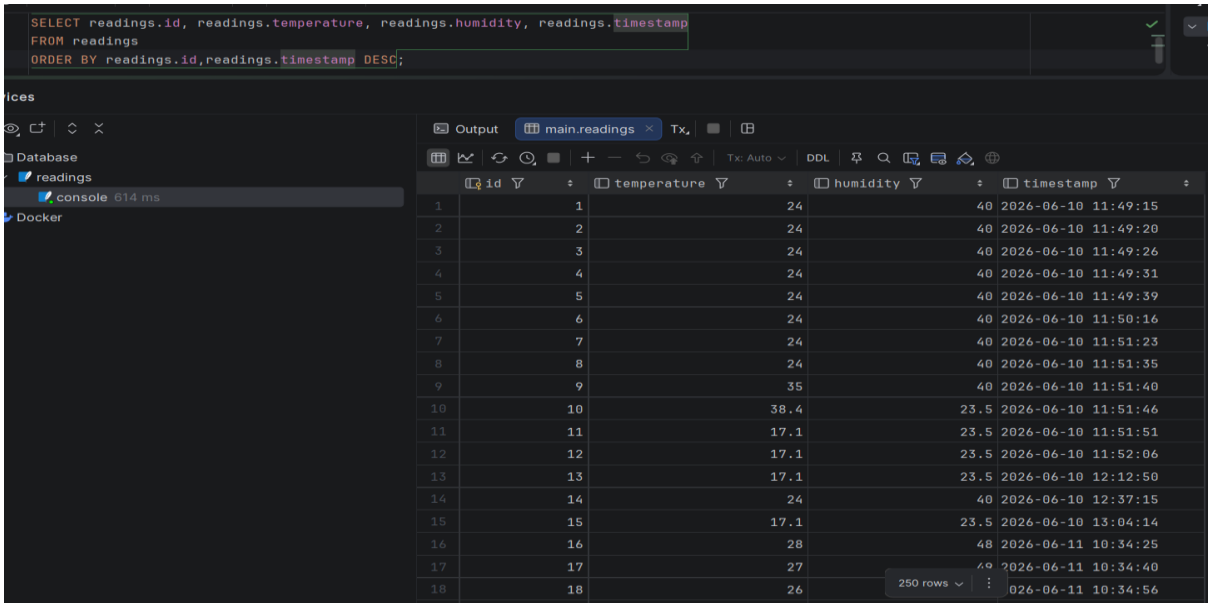
Below image shows some of the code snippets from the terminal to install the initial libraries needed to connect and detect the TTGO board on the laptop

Figure 6 Installing Required Libraries

Database Connection from the init_db.py file use to create the database in SQLITE .

```python
#ICS GROUP 4D GROUP 1 EMBEDDED SYSTEMS AND IOT
import sqlite3

DB_FILE = "readings.db"


def create_database():
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()

    cursor.execute(
        """
        CREATE TABLE IF NOT EXISTS readings (
            id          INTEGER PRIMARY KEY AUTOINCREMENT,
            temperature REAL,
            humidity    REAL,
            timestamp   TEXT
        )
        """
    )

    conn.commit()
    conn.close()
    print("OK: database '{}' is ready with table 'readings'.".format(DB


if __name__ == "__main__":
    create_database()
```

Figure 7 Creating and Connecting to the Database



	id	temperature	humidity	timestamp
1	1	24	40	2026-06-10 11:49:15
2	2	24	40	2026-06-10 11:49:20
3	3	24	40	2026-06-10 11:49:26
4	4	24	40	2026-06-10 11:49:31
5	5	24	40	2026-06-10 11:49:39
6	6	24	40	2026-06-10 11:50:16
7	7	24	40	2026-06-10 11:51:23
8	8	24	40	2026-06-10 11:51:35
9	9	35	40	2026-06-10 11:51:40
10	10	38.4	23.5	2026-06-10 11:51:46
11	11	17.1	23.5	2026-06-10 11:51:51
12	12	17.1	23.5	2026-06-10 11:52:06
13	13	17.1	23.5	2026-06-10 12:12:50
14	14	24	40	2026-06-10 12:37:15
15	15	17.1	23.5	2026-06-10 13:04:14
16	16	28	48	2026-06-11 10:34:25
17	17	27	49	2026-06-11 10:34:40
18	18	26		026-06-11 10:34:56

Figure 8 Querying and Reading The Data Received From The Sensor

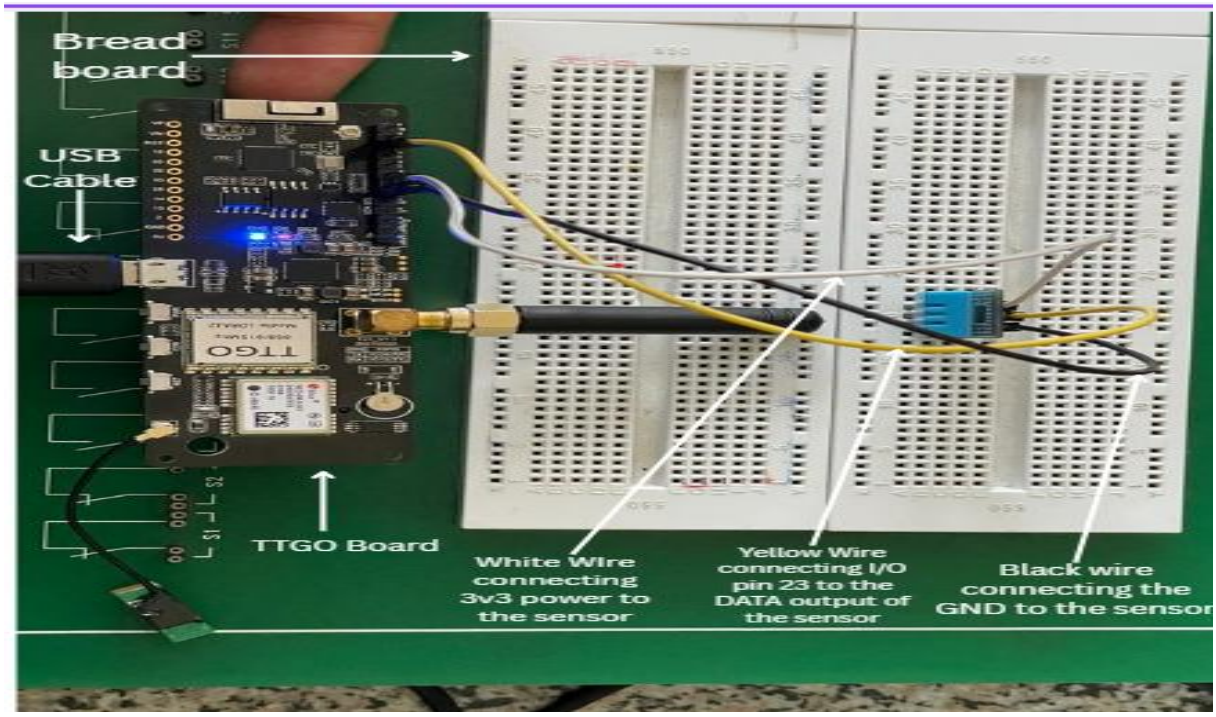


Figure 9 The actual Prototype of the TTGO and DHT22